PROJECT OVERVIEW

Objective

To design and develop a **C++-based machine learning engine** optimized for **edge inference**, with support for **on-device learning**, enabling intelligent power consumption prediction on embedded edge controllers.

Tools

- **STM32CubeIDE** – Embedded firmware development
- **STM32CubeMX** – Peripheral and project configuration
- **STM32 Nucleo-F429ZI** – Target development platform
- **CMSIS-NN** – Optimized neural network kernels for ARM Cortex-M processors

Stages of Development

1. **Python implementation of the power prediction algorithm with pre-existing frameworks**
2. **Development of a tensor-based automatic differentiation (backpropagation) engine**
3. **Implementation of gradient descent-based training**
4. **Integration of model optimization techniques**
    a. Self-pruning
    b. Quantization
5. **Integration with CMSIS-NN** for hardware-optimized inference on ARM Cortex-M microcontrollers
6. **Deployment of a power consumption prediction model** on the embedded target

Results and Benefits

1. **Predictive Power Consumption:** By forecasting the next day's power consumption, diesel procurement and dispatch for diesel generators (DGs) can be planned more efficiently, reducing unnecessary transportation, storage, and fuel procurement costs.

2. **Reusable Machine Learning Engine:** The developed C++ machine learning engine is designed to be modular and reusable, allowing it to be adapted for applications beyond power consumption prediction wherever lightweight AI models are required on embedded edge devices.
3. **Edge Deployment:** The engine is optimized for resource-constrained embedded systems, enabling low-latency inference without relying on cloud connectivity.
4. **Support for On-Device Learning:** The architecture provides the foundation for on-device model updates, allowing edge controllers to adapt to changing operating conditions over time.

THEORY

Inference

Inference is the process of using a trained neural network to make predictions.

A neural network can be viewed as a large mathematical function composed of many interconnected operations. During inference, an input is passed through this function to produce an output without modifying any of the model parameters.

Backpropagation

Backpropagation is the process of estimating the gradient of each model parameter with respect to the final loss.

In machine learning, after computing a loss value, gradients are propagated backward through the computational graph using the chain rule. These gradients indicate how much each parameter contributed to the final error.

Example (Power Consumption Prediction):

Suppose power consumption data is collected for **60 days**.

- The model predicts power consumption for each day.
- The prediction error (loss) is computed every day.
- The losses are summed or averaged to obtain a final cost.
- Backpropagation computes the gradient of every model parameter with respect to this final cost, indicating how each parameter should be adjusted to reduce the prediction error.

Training

Training is the process of adjusting the model parameters to minimize overall loss.

This is achieved by repeatedly performing:

1. Forward propagation (prediction)
2. Loss computation
3. Backpropagation
4. Parameter update

The process continues until the model learns parameters that produce accurate predictions.

Gradient Descent

Gradient descent is an optimization algorithm used during training.

It utilizes the gradients computed through backpropagation to update the model parameters in the direction that reduces the loss. Repeating this process over multiple iterations gradually improves the model's performance.

Pruning

Pruning is a model optimization technique that removes parameters, contributing very little to the model's output.

Typically, weights whose values are very close to zero have minimal influence on the final prediction. Removing these weights reduces memory usage and computational complexity while maintaining nearly the same prediction of accuracy.

Quantization

Quantization is a model optimization technique that reduces the precision used to represent model parameters.

Instead of storing weights as high-precision floating-point numbers (e.g., 32-bit floating point), they are represented using lower-precision formats (such as 8-bit integers). This significantly reduces memory consumption and improves inference speed on embedded hardware, with only a small reduction in model accuracy.

CMSIS-NN

CMSIS-NN is a collection of highly optimized neural network kernels developed for ARM Cortex-M microcontrollers.

Integrating the machine learning engine with CMSIS-NN enables faster inference and lower power consumption by utilizing hardware-optimized implementations of common neural network operations.

PROJECT PROGRESS AND MILESTONES

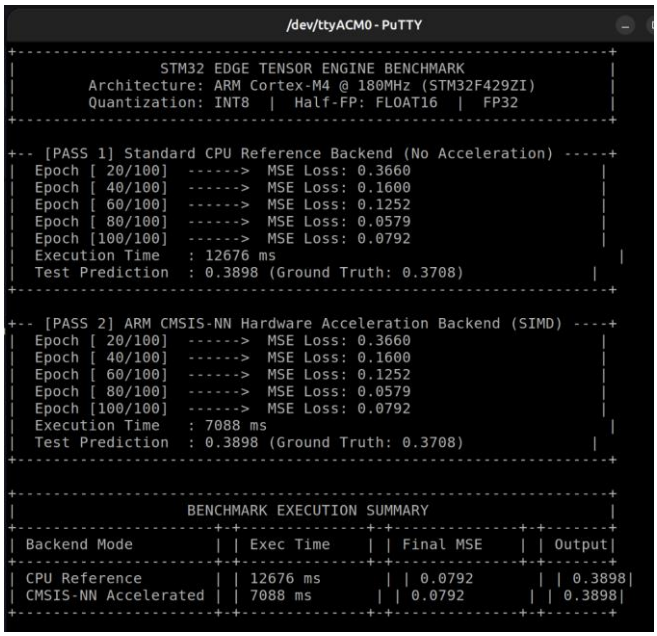
#1 JULY 7

- Crude implementation of autograd engine
- Using dynamic node building
- Result: shows possibility of basic edge regression tasks

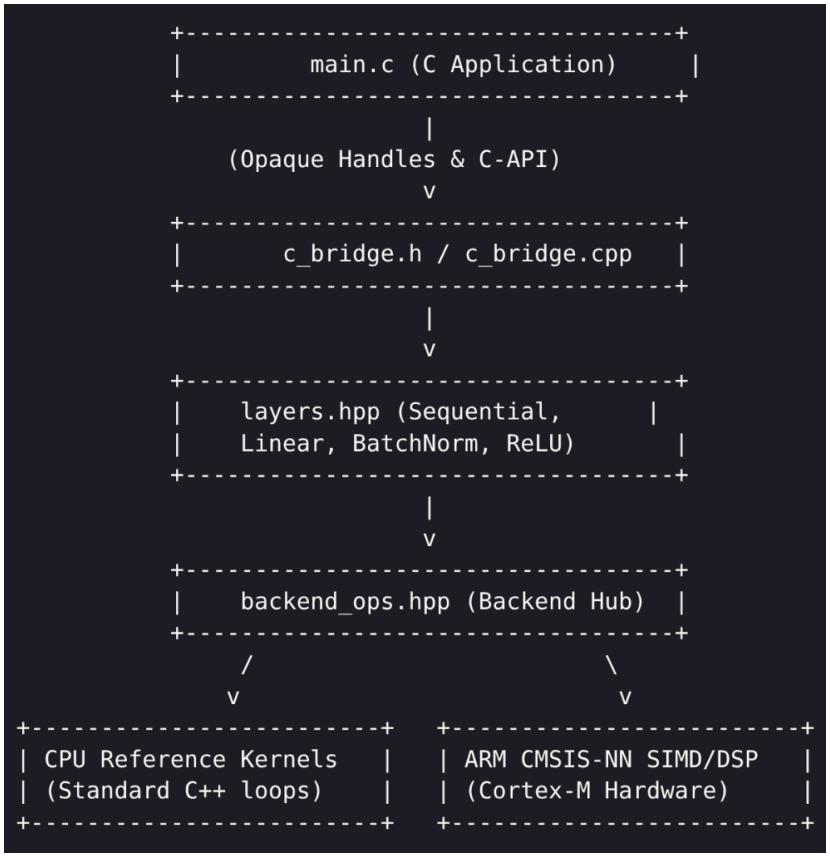
#2 JULY 27

- Switched to Static layer topology
- No Dynamic Node Allocation: During execution, no Node or Edge objects are created or freed on the heap per operation.
- Static Reverse-Pass Traversal: Backpropagation does not traverse a dynamically constructed operation graph.
- The above removes/add obstructions to the possibility of on device pruning
- However, results in much lower memory footprint
- Added CMSIS_NN backend for vectorization and faster execution
- Cmsis_nn reduced training time of 100 epoch from 12ms to 7ms
- Added int8 quantization and fp16 data type
- Added tanh, exp, softmax layers and operations

- Benchmark:



- System Architecture



- Current repository structure

```
tensor_engine/
├── Core/
│   ├── Inc/
│   │   └── main.h
│   └── Src/
│       ├── main.c                          # Entry point & dual-pass benchmark suite
│       └── engine/
│           ├── memory/memory.hpp           # MemoryBuffer, WeightBuffer, Float16 &
Quantization
│           ├── array_api/array_api.hpp     # Matrix / Tensor container definitions
│           ├── ops/
│           │   ├── ops.hpp                 # High-level matrix operation wrappers
│           │   └── backend_ops.hpp         # Hardware backend dispatch & CMSIS-NN routines
│           ├── autograd/autograd.hpp       # GradTensor & AutogradNode for operation-level
graph
│           ├── layers/layers.hpp           # Linear, BatchNorm, LeakyReLU, Dropout,
Sequential
│           └── bridge/
│               ├── c_bridge.h              # C-API header for STM32CubeIDE
│               └── c_bridge.cpp            # C-API wrapper implementation
├── Drivers/
│   └── CMSIS-NN/                          # ARM CMSIS-NN library source files & headers
└── README.md
```

Further Workings

1. Local serialization and deserialization of neural network architectures.
2. Conversion of the framework into a reusable static library.
3. Python-based neural network architecture definition with export capabilities. (for remote pretraining and pruning)